SANDIP FOUNDATION'S
SANDIP INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE
(An Autonomous Institute Permanently Affiliated to SPPU, Pune)
MAHIRAVANI, TRIMBAK ROAD, TAL & DIST: NASHIK-422213, MAHARASHTRA,INDIA

Sem-V	2410305: Computer Graphics Lab		
Teaching Scheme:	**Credits**	**Examination Scheme**	
Theory:--	**Th:--**	**Theory**	**--**
Practical:4 hrs./week	**Practical:01**		--
Prerequisite: A basic understanding of computer graphics concepts, 2D/3D transformations, and proficiency in a programming language like C or C++ (sometimes Java or Python).		**Practical**	**25**
		Oral	--
		Term work	**--**

Course Objectives: The student should be able to

1. To acquaint the learners with the concepts of OpenGL.
2. To acquaint the learners with the basic concepts of Computer Graphics.
3. To implement various algorithms for generating and rendering graphical objects.
4. To get familiar with the mathematics behind graphical transformations.
5. To understand and apply various methods and techniques related to animation.

Course Outcomes: On completion of the course, learner will be able to–

CO	BT LEVEL
CO1: Apply line and circle drawing algorithms to construct graphical objects.	**L3**
CO2: Apply polygon filling methods to render objects accurately.	**L3**
CO3: Apply polygon clipping algorithms to manage object visibility.	**L3**
CO4: Apply 2D transformations such as translation, rotation, scaling on graphical objects.	**L3**

CO-PO_PSO Mapping

Course Outcomes	PO's												PSO		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	3	3	2	3	–	–	–	1	–	–	2	3	3	2
CO2	3	3	3	2	3	–	–	–	1	–	–	2	3	3	2
CO3	3	3	3	3	3	–	–	–	1	–	–	2	3	3	3
CO4	3	3	3	2	3	–	–	–	1	–	–	2	3	3	2

Sr.No.	Title of Experiment (Any 8)	CO
1	Install and explore the OpenGL	CO1
2	Implement DDA and Bresenham line drawing algorithm to draw: i)Simple Line ii)Dotted Line iii)Dashed Line iv) Solid line	CO1
3	Using mouse interface Divide the screen in four quadrants with center as (0,0).The line should work for all the slopes positive as well as negative.	CO1
4	Implement Bresenham circle drawing algorithm to draw any object. The object should be displayed in all the quadrants with respect to center and radius	CO2
5	Clipping & Windowing algorithms for points, lines & polygons	CO2
6	Implement Cohen Suther land polygon clipping method to clip the polygon with respect the viewport and window. Use mouse click, keyboard interface	CO3
7	2-D/3-D transformations	CO3
8	Implement following 2D transformations on the object with respect to axis: i) Scaling ii) Rotation about arbitrary point iii) Reflection	CO4
9	Simple fractals representation	CO3
10	Generate fractal patterns using i)Bezier ii)Koch Curve	CO4

Text Books

1. Hearn, Baker——Computer Graphics(C version2ndEd.)‖–Pearson education
2. Z.Xiang,R. Plastock——Schaum'soutlines Computer Graphics(2ndEd.)‖ –TMH
3. D.F. Rogers,J.A. Adams——Mathematical Elements for Computer Graphics(2ndEd.)‖– TMH
4. Mukherjee, Fundamentals of Computer graphics & Multimedia, PHI
5. Sanhker, Multimedia–A Practical Approach, Jaico

ReferenceBooks

1. Computer Graphics Principle and Practice, J.D.Foley, A.Dam, .K.Feiner, Addison Wesley.
2. Procedural Elements of Computer Graphics, David Rogers, TMH.
3. Computer Graphics: Algorithms and Implementations, D.PMukherjee, D.Jana, PHI.
4. Computer Graphics, Z.Xiang, R.A.Plastock, Schaum's Outlines, McGrow Hill.

Online Learning Resources & Material links:

1. **GitHub:** https://github.com/terkelg/awesome-creative-coding
   https://github.com/ssloy/tinyrenderer
2. **SWAYAM-NPTEL:** https://onlinecourses.nptel.ac.in/noc24_cs82/preview
   https://onlinecourses.nptel.ac.in/noc20_cs90/preview
3. **NPTEL:** https://archive.nptel.ac.in/courses/106/106/106106090/

Practical No. 1

Title: Install and explore the OpenGL.

Objectives:

- To acquaint the learners with the concepts of OpenGL and familiar with Graphics Environment

Brief Theory:

OpenGL is a well-known standard for generating 3-D as well as 2-D graphics that is extremely powerful and has many capabilities. OpenGL is defined and released by the OpenGL Architecture Review Board (ARB). The latest version of OpenGL at the time of this writing is 4.4, which uses a different technique for drawing. OpenGL is a low-level, widely supported modelling and rendering software package, available across all platforms. It can be used in a range of graphics applications, such as games, CAD design, or modelling. OpenGL API is designed mostly in hardware.

- **Design**: This API is defined as a set of functions which may be called by the client program. Although functions are similar to those of C language but it is language independent.
- **Development**: It is an evolving API and Khronos Group regularly releases its new version having some extended feature compare to previous one. GPU vendors may also provide some additional functionality in the form of extension.
- **Associated Libraries**: The earliest version is released with a companion library called OpenGL utility library. But since OpenGL is quite a complex process. So in order to make it easier other library such as OpenGL Utility Toolkit is added which is later superseded by freeglut. Later included library were GLEE, GLEW and glbinding.
- **Implementation**: Mesa 3D is an open source implementation of OpenGL. It can do pure software rendering and it may also use hardware acceleration on BSD, Linux, and other platforms by taking advantage of Direct Rendering Infrastructure.
-

1.2 Installation of OpenGL on Ubuntu:

- step 1: sudo apt-get update
- step 2: sudo apt-get install libglu1-mesa-dev freeglut3-dev mesa-common-dev

1.3 Program execution on Ubuntu with OpenGL:

- $ g++ Traingle.cc -lglut -lGL -o t1
- $ ./t1

1.4 OpenGL Pipeline

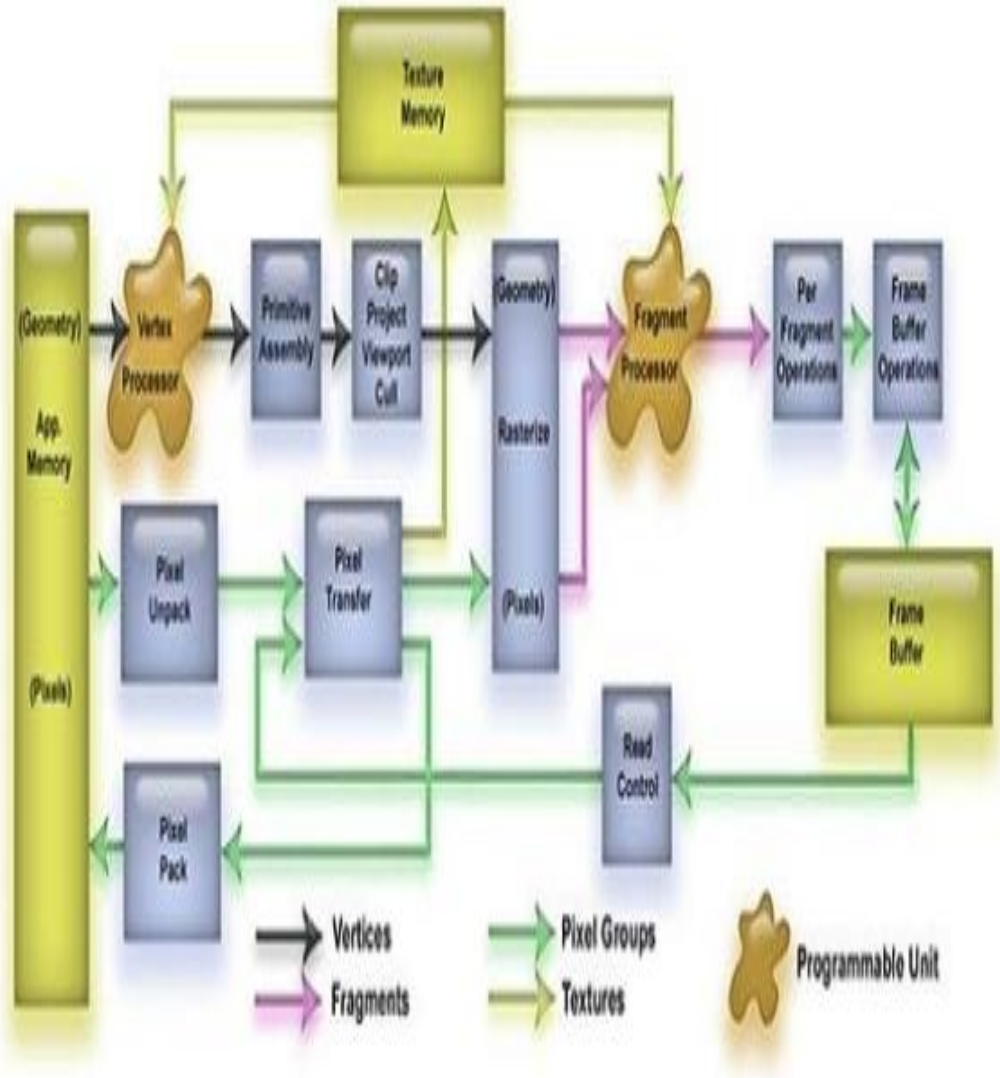


Figure 1. OpenGL Architecture

Figure shows the programmable OpenGL pipeline with the vertex and fragment processors. As we can imagine, the OpenGL pipeline is complex, but we do not have to understand it fully in order to be able to use OpenGL. The Pipeline shows how OpenGL operates in the background. Newer versions of the OpenGL Pipeline are even more complex! OpenGL is a big state machine. Most calls to OpenGL functions modify a global state that we cannot access directly. The OpenGL Shading Language code that is intended for execution on one of the OpenGL programmable processors is called a Shader. The OpenGL Shading Language has its roots in C (presenting the OpenGL Shading Language is beyond the scope of this article). OpenGL does not define windowing layer, because it tries to be platform-neutral and leaves this functionality to the operating system. The OS must provide a rendering context that accepts commands as well as a frame buffer that keeps the results of the drawing commands.

1.5 Built-in functions:

i. **Window title:** The WINDOW_TITLE_PREFIX pre-processor definition simply serves as a placeholder for our window title. The reason it's called prefix is because we'll be appending some dynamic text to it. We also define the global variable WindowHandle, used to store the handle to the window created by FreeGLUT.

ii. **Initializing:** After declaring the function prototypes and defining the main application entry function, the next function of interest is the InitWindow function, called by the Initialize function. InitWindow calls several FreeGLUT functions to create a window, starting off with glutInit, which initializes the FreeGLUT library. This function does not return anything, but simply sets up FreeGLUT for use in your application. In any program, this should be the first FreeGLUT function you should call. The glutInit takes two arguments, which are the command line arguments from the main function.

iii. **Context Types and Window Options**: The next three function calls go together as they declare to FreeGLUT what type of OpenGL context we'd like to use for our program.

- glutInitContextVersion
- glutInitContextFlags
- glutInitContextProfile

The glutSetOption is called with the GLUT_ACTION_ON_WINDOW_CLOSE option to make sure that the glutMainLoop function in main returns to the program and doesn't exit the program when it's done. The original GLUT library did not return to the program and after GLUT was done rendering it would end your program. This meant that if you had allocated any memory, there was no way to avoid memory leaks without some nasty hacking into the GLUT library. FreeGLUT effectively ends this behaviour by setting the GLUT_ACTION_ON_WINDOW_CLOSE option to GLUT_ACTION_GLUTMAINLOOP_RETURNS. GlutInitWindowSize specifies the initial size

of the rendering window, in our case the values stored in the CurrentHeight and CurrentWidth variables, 800 and 600 respectively.

iv. **Display Modes:** The glutInitDisplayMode is a function that is a bit more involved since it defines what type of OpenGL context we would like and how the device should render our scene. **Generally**, three distinct options are passed to the function, namely:

GLUT_DEPTH enables the usage of the depth buffer, an important mechanism in 3D computer graphics. The depth buffer (also called Z-buffer) contains the floating point Z-depth information of each pixel rendered to the screen**.**

GLUT_DOUBLE is a flag that enables the usage of double-buffering, which is a feature that reduces image flickering. With double buffering, all of the draw commands are executed on an off-screen buffer, which is sent to the screen when all of the drawing for a frame has been completed so that no incomplete images are displayed.

GLUT_RGBA is a flag that defines the way colours are composited by using individual Red, Green, Blue, and Alpha values. GlutInitDisplayMode can take several other options, which we'll explore throughout the book.

v. **Window Creation:** A call to glutCreateWindow will create our context and will return a handle to our newly created window. The sole parameter to this function is WINDOW_TITLE_PREFIX, which will set the initial title for the window. A FreeGLUT window is not valid if the window-handle's value is less than the number 1, at which point the program should fail or retry to create the window.

vi. **Function Call-Backs:** To handle window resizing and rendering the scene, which we'll set with glutReshapeFunction and glutDisplayFunction, respectively. The reshape function is called each time the window is resized, and the display function is called each time the scene is to be drawn to the screen.

vii. **Debug Output:** InitWindow is finished at this point and returns to the Initialize function.

viii. **Screen Clear Color:** The last function that we'll call in the Initialize function is glClearColor with its four parameters set to 0.0f, representing (in order) the Red, Green, Blue, and Alpha color channels, this corresponds to the FreeGLUT flag GLUT_RGBA that we used with the glutInitDisplayMode function. The values provided to glClearColor must be on a scale from 0.0f to 1.0f, corresponding to 0% - 100% color channel intensity. The RGB color mode is the primary method used in displaying colors on many electronic systems, and also in OpenGL. The glClearColor function is used to define the color with which to clear the back buffer each time a frame is prepared to be drawn to the screen. Example:

- glClearColor(1.0f, 0.0f, 0.0f, 0.0f); for red
- glClearColor(0.0f,1.0f, 0.0f, 0.0f); for green
- glClearColor(0.0f, 0.0f,1.0f, 0.0f); for blue
- glClearColor(1.0f, 0.0f,1.0f, 0.0f); for purple

- glClearColor(0.0f, 0.0f,0.5f, 0.0f); for dark blue

1.6 Developer-Driven Advantages

- **Industry standard:** An independent consortium, the OpenGL Architecture Review Board, guides the OpenGL specification. With broad industry support, OpenGL is the only truly open, vendor-neutral, multiplatform graphics standard.
- **Stable:** OpenGL implementations have been available for more than seven years on a wide variety of platforms. Additions to the specification are well controlled, and proposed updates are announced in time for developers to adopt changes. Backward compatibility requirements ensure that existing applications do not become obsolete.
- **Reliable and portable**: All OpenGL applications produce consistent visual display results on any OpenGL API-compliant hardware, regardless of operating system or windowing system.
- **Evolving:** Because of its thorough and forward-looking design, OpenGL allows new hardware innovations to be accessible through the API via the OpenGL extension mechanism. In this way, innovations appear in the API in a timely fashion, letting application developers and hardware vendors incorporate new features into their normal product release cycles.
- **Scalable:** OpenGL API-based applications can run on systems ranging from consumer electronics to Pcs, workstations, and supercomputers. As a result, applications can scale to any class of machine that the developer chooses to target.
- **Easy to use:** OpenGL is well structured with an intuitive design and logical commands. Efficient OpenGL routines typically result in applications with fewer lines of code than those that make up programs generated using other graphics libraries or packages. In addition, OpenGL drivers encapsulate information about the underlying hardware, freeing the application developer from having to design for specific hardware features.

Well-documented: Numerous books have been published about OpenGL, and a great deal of sample code is readily available, making information about OpenGL inexpensive and easy to obtain.

Conclusion

Thus we have studied and learned the concept of OPENGL .

Practical No. 2

Title:

Implementation of DDA and Bresenham Line Drawing Algorithms to draw:
(i) Simple Line
(ii) Dotted Line
(iii) Dashed Line
(iv) Solid Line

Objective:

To implement and compare the DDA and Bresenham line drawing algorithms for different line styles (simple, dotted, dashed, and solid) using C++.

Theory:

In computer graphics, line drawing algorithms are used to determine which pixels should be plotted to form a close approximation to a straight line between two points.

1. DDA (Digital Differential Analyzer) Algorithm

The DDA algorithm calculates intermediate positions along the line between two endpoints by incrementally adding a fixed amount to either the x or y coordinate.

- **Advantages:**
    - Simple and easy to implement.
- **Disadvantages:**
    - Requires floating-point operations.
    - Slower than Bresenham due to rounding.

2. Bresenham's Line Drawing Algorithm

Bresenham's algorithm uses integer arithmetic to decide which pixel is the closest to the ideal line at each step.

- **Advantages:**
    - Faster due to integer calculations.
    - More efficient for raster displays.

Line Styles

- **Simple Line:** Continuous line between two endpoints.
- **Dotted Line:** Plot every alternate pixel.
- **Dashed Line:** Plot a few pixels, then skip a few, repeatedly.
- **Solid Line:** Same as a simple line (all pixels plotted).

Algorithm:

1. **DDA Algorithm**
   1. Input two endpoints (x0, y0) and (x1, y1).
   2. Calculate dx = x1 - x0 and dy = y1 - y0.
   3. Determine steps = max(|dx|, |dy|).
   4. Calculate increments:

Xinc = dx / steps
Yinc = dy / steps

5.Initialize (x, y) = (x0, y0).

6.For each step, plot (round(x), round(y)), then increment:

x = x + Xinc
y = y + Yinc

7. Repeat until the end point is reached.

2. Bresenham Algorithm

1. Input two endpoints (x0, y0) and (x1, y1).
2.Initialize:

dx = |x1 - x0|, dy = |y1 - y0|
sx = sign(x1 - x0), sy = sign(y1 - y0)
err = dx - dy

3.Plot (x0, y0).

4.While (x0 != x1) or (y0 != y1):

Plot (x0, y0)

Calculate e2 = 2 * err

If e2 > -dy, update err -= dy; x0 += sx

If e2 < dx, update err += dx; y0 += sy

5. Repeat until the end point is reached.

Conclusion

The program successfully implements both DDA and Bresenham line drawing algorithms to draw simple, dotted, dashed, and solid lines.

Practical No. 3

Title:

Using a Mouse Interface, Divide the Screen into Four Quadrants with the Center as (0,0). Draw a Line for All Slopes, Both Positive and Negative.

Aim

Graphics screen into four quadrants, with the origin (0,0) at the center, and draw lines with positive, negative, horizontal, and vertical slopes.

Objectives

1. To divide the graphics window into four quadrants using X and Y axes.
2. To place the origin (0,0) at the center of the screen.
3. To accept two endpoints using the mouse interface.
4. To implement the DDA line-drawing algorithm.
5. To draw lines with positive, negative, horizontal, and vertical slopes.
6. To convert mouse screen coordinates into Cartesian coordinates.

Theory

In computer graphics, a line-drawing algorithm is used to determine the pixels required to display a straight line between two points.

In this practical, OpenGL is used to create a graphics window, and the mouse is used to select the starting and ending points of a line.

The graphics window is divided into four quadrants using the X-axis and Y-axis. The origin (0,0) is positioned at the center of the window.

Coordinate System

- First Quadrant (I): (+x, +y)
- Second Quadrant (II): (-x, +y)
- Third Quadrant (III): (-x, -y)
- Fourth Quadrant (IV): (+x, -y)

Mouse Interface

The user selects two points by clicking the left mouse button. The first click specifies the starting point, and the second click specifies the ending point. The program converts the mouse coordinates into Cartesian coordinates and draws a line between the two points.

DDA Line-Drawing Algorithm

The Digital Differential Analyzer (DDA) algorithm is used to draw a straight line by calculating intermediate points between two endpoints.

The number of steps is determined by the maximum absolute difference between the X and Y coordinates.

The X and Y increments are calculated using the following formulas:

dx = x2 - x1

dy = y2 - y1

steps = max(|dx|, |dy|)

X increment = dx / steps

Y increment = dy / steps

The algorithm supports lines with positive and negative slopes, as well as horizontal and vertical lines.

Algorithm

1. Start the program.
2. Initialize the OpenGL and FreeGLUT environment.
3. Create a graphics window.
4. Calculate the center of the window.
5. Draw the X-axis and Y-axis to divide the screen into four quadrants.
6. Display the origin (0,0) at the center.
7. Register the mouse callback function.
8. Wait for the user to click the left mouse button.
9. Store the first mouse click as the starting point.
10. Wait for the second mouse click and store it as the ending point.
11. Convert both mouse coordinates into Cartesian coordinates.
12. Calculate dx, dy, and the number of steps.
13. Apply the DDA algorithm to calculate the intermediate points.
14. Draw the line between the two selected points.
15. Display the line on the graphics window.
16. Allow the user to draw another line.
17. Press the Escape key to terminate the program.
18. Stop.

A graphics window appears with the X-axis and Y-axis intersecting at the center (0,0).

- Click once to select the starting point.
- Click again to select the ending point.
- A red line is drawn between the two selected points.
- The program supports all four quadrants and all line slopes.
- Press **C** to clear the line and draw another.
- Press **Esc** to exit.

Result

The program successfully divides the graphics window into four quadrants, with the origin (0,0) at the center. It accepts two points through the mouse interface and draws a line between them using the DDA algorithm. The program supports positive, negative, horizontal, and vertical slopes.

Conclusion

Thus, the mouse-based line-drawing program was successfully implemented using C++ and OpenGL. The screen was divided into four quadrants, and lines with different slopes were drawn by selecting endpoints using the mouse.

Practical No. 4

Title:

Implementation of Bresenham's Circle Drawing Algorithm to Draw an Object. The Object Should Be Displayed in All the Quadrants with Respect to Centre and Radius.

Aim

To implement Bresenham's circle drawing algorithm using C++ and OpenGL and to draw an object (a target made of concentric circles) for a given centre and radius, so that it is displayed in all four quadrants.

Objectives

1. To understand the eight-way symmetry of a circle.
2. To implement Bresenham's circle drawing algorithm using only integer arithmetic.
3. To accept the centre (xc, yc) and the radius r from the user.
4. To draw an object using concentric circles drawn with respect to the given centre and radius.
5. To display the object in all four quadrants with the origin (0,0) at the centre of the screen.

Theory

A circle with centre (xc, yc) and radius r is the set of points that satisfy the equation $(x - xc)^2 + (y - yc)^2 = r^2$. Plotting the circle directly from this equation needs square roots, and plotting it with sin and cos needs trigonometric functions. Both methods are slow and use floating-point arithmetic. Bresenham's circle algorithm avoids this by choosing, at every step, the pixel closest to the true circle with the help of an integer decision parameter.

Eight-Way Symmetry

A circle is symmetric about both axes and about the lines y = x and y = -x. So if the point (x, y) is calculated for a circle with centre at the origin, seven more points are obtained without extra calculation. For a circle with centre (xc, yc), the eight points are:

- (xc + x, yc + y) and (xc - x, yc + y)
- (xc + x, yc - y) and (xc - x, yc - y)
- (xc + y, yc + x) and (xc - y, yc + x)
- (xc + y, yc - x) and (xc - y, yc - x)

Therefore only one octant (from x = 0 to x = y) has to be calculated.

Bresenham's Circle Algorithm

The algorithm starts at the point (0, r) and moves clockwise along the circle. At every step x is increased by 1, and the decision parameter d decides whether y stays the same or decreases by 1.

- Initial decision parameter: d0 = 3 - 2r
- If d < 0: the next pixel is (x + 1, y) and d = d + 4x + 6
- If d >= 0: the next pixel is (x + 1, y - 1) and d = d + 4(x - y) + 10

- The loop is repeated while x <= y.

Object Drawn in this Practical

The object is a target made of four concentric circles with radii r, 3r/4, r/2 and r/4. All the circles use the same centre, so the object scales automatically with the radius entered by the user. The origin (0,0) is placed at the centre of the window, so a circle drawn around the origin lies in all four quadrants.

Algorithm

1. Start the program.
2. Read the centre (xc, yc) and the radius r.
3. Initialize the OpenGL and FreeGLUT environment and create the window.
4. Set the coordinate system so that the origin (0,0) is at the centre of the window, and draw the X-axis and Y-axis.
5. Set x = 0, y = r and the decision parameter d = 3 - 2r.
6. While x <= y, plot the eight symmetric points (xc ± x, yc ± y) and (xc ± y, yc ± x).
7. If d < 0, set d = d + 4x + 6. Otherwise set d = d + 4(x - y) + 10 and decrease y by 1.
8. Increase x by 1 and repeat steps 6 and 7 until x > y.
9. Repeat steps 5 to 8 for the radii 3r/4, r/2 and r/4 with different colours to draw the object.
10. Display the object on the screen.
11. Stop.

Result

The program draws the circles using Bresenham's algorithm with integer calculations only. The object is displayed correctly in all the quadrants for the given centre and radius.

Conclusion

Thus, Bresenham's circle drawing algorithm was successfully implemented using C++ and OpenGL. Eight-way symmetry reduces the work to one octant, and the integer decision parameter makes the algorithm fast and free of rounding errors.

Practical No. 5

Title:

Implementation of Clipping and Windowing Algorithms for Points, Lines and Polygons.

Aim

To implement point clipping, Cohen–Sutherland line clipping and Sutherland–Hodgman polygon clipping against a rectangular clipping window using C++ and OpenGL.

Objectives

1. To understand the concept of a clipping window and the need for clipping.
2. To implement point clipping using the window boundary test.
3. To implement the Cohen–Sutherland line clipping algorithm using region codes.
4. To implement the Sutherland–Hodgman polygon clipping algorithm.
5. To display the objects before and after clipping.

Theory

Clipping is the process of removing the part of a picture that lies outside a specified region called the clipping window. Only the part inside the window is displayed. In this practical the clipping window is the rectangle with corners (xmin, ymin) = (-100, -100) and (xmax, ymax) = (100, 100). Windowing means selecting the part of the world to be viewed through this window.

1. Point Clipping

A point (x, y) is displayed only if xmin <= x <= xmax and ymin <= y <= ymax. Otherwise it is discarded.

2. Cohen–Sutherland Line Clipping

Every end point of the line is given a 4-bit region code. Bit 1 is set if the point is to the left of the window (x < xmin), bit 2 if it is to the right (x > xmax), bit 3 if it is below (y < ymin) and bit 4 if it is above (y > ymax). A point inside the window has the code 0000.

- Trivial accept: both end codes are 0000 (code1 OR code2 = 0). The line is drawn as it is.
- Trivial reject: the logical AND of the two codes is not zero. Both end points are on the outside of the same boundary, so the line is discarded.
- Otherwise one end point that is outside is moved to the intersection of the line with a window boundary, and the test is repeated.
- Intersection with y = ymax: x = x0 + (x1 - x0)(ymax - y0)/(y1 - y0). Intersection with x = xmax: y = y0 + (y1 - y0)(xmax - x0)/(x1 - x0). The other two boundaries are found in the same way.

3. Sutherland–Hodgman Polygon Clipping

The polygon is clipped against one window boundary at a time (left, right, bottom, top). The output polygon of one boundary is the input of the next. For every edge from the previous vertex P to the current vertex C, one of four cases occurs:

- P outside, C inside: output the intersection point and C.
- P inside, C inside: output C.
- P inside, C outside: output the intersection point.
- P outside, C outside: output nothing.

Algorithm

Point clipping

1. Read the point (x, y).
2. If xmin <= x <= xmax and ymin <= y <= ymax, display the point. Otherwise discard it.

Cohen–Sutherland line clipping

1. Read the end points (x0, y0) and (x1, y1) of the line.
2. Calculate the region codes code0 and code1 of the two end points.
3. If (code0 OR code1) = 0, accept the line and go to step 7.
4. If (code0 AND code1) is not 0, reject the line and stop.
5. Select an end point that is outside the window and find the intersection of the line with the boundary indicated by its code.
6. Replace that end point by the intersection point, calculate its new code and go to step 3.
7. Draw the (clipped) line.

Sutherland–Hodgman polygon clipping

1. Store the vertices of the polygon in an input list.
2. For each window boundary (left, right, bottom, top) create an empty output list.
3. For each edge of the input list, apply the four cases given in the theory and add the resulting points to the output list.
4. Make the output list the input list for the next boundary.
5. After the four boundaries have been processed, draw the polygon formed by the final list.

Result

The program successfully clips points, lines and polygons against the rectangular window. The original objects are shown in grey and the visible parts in red.

Conclusion

Thus, point clipping, Cohen–Sutherland line clipping and Sutherland–Hodgman polygon clipping were successfully implemented using C++ and OpenGL. Region codes allow most lines to be accepted or rejected without calculating intersections, and the polygon is clipped one boundary at a time.

Practical No. 6

Title:

Implementation of the Cohen–Sutherland Type (Sutherland–Hodgman) Polygon Clipping Method to Clip a Polygon with Respect to the Viewport and Window. Use Mouse Click and Keyboard Interface.

Aim

To clip a polygon entered with the mouse against a clipping window using the Sutherland–Hodgman method, and to display the clipped polygon in a viewport using the window to viewport transformation. The mouse is used to enter the polygon and the keyboard is used to control the program.

Objectives

1. To accept the vertices of a polygon using mouse clicks.
2. To implement the polygon clipping method that uses the inside/outside tests of the Cohen–Sutherland approach.
3. To perform the window to viewport transformation.
4. To use the keyboard to clip, reset and move the clipping window.
5. To display the original polygon in the window and the clipped polygon in the viewport.

Theory

The Cohen–Sutherland algorithm clips a single line segment. A polygon is a closed chain of edges, and after clipping it must still be a closed polygon, so clipping each edge separately is not enough. The Sutherland–Hodgman method therefore applies the same inside/outside test against each window boundary in turn and produces a new vertex list that forms a closed polygon. The method is exact for convex polygons. For a concave polygon the result may contain extra edges along the window boundary.

Window and Viewport

The window is the area of the world coordinate system that is selected for display. The viewport is the area of the screen in which the selected part is displayed. In this practical the left half of the screen is the window area and the right half contains the viewport.

Mouse and Keyboard Interface

- Left mouse button: adds a vertex of the polygon inside the window area.
- Right mouse button: closes the polygon (at least three vertices are needed).
- Key c: clips the polygon.
- Key r: removes the polygon so that a new one can be entered.
- Arrow keys: move the clipping window by 10 units, and the polygon is clipped again.
- Esc key: exits the program.

Algorithm

1. Start the program and create the graphics window of size 800 × 400.
2. Define the clipping window in the left half and the viewport in the right half of the screen.
3. Register the mouse, keyboard and special key (arrow key) callback functions.
4. On each left click, convert the mouse position to window coordinates (y = height - y) and store it as a vertex.
5. On a right click, close the polygon.
6. When the key c is pressed, clip the polygon against the left, right, bottom and top boundaries of the window using the Sutherland–Hodgman method.
7. Draw the original polygon in black and the clipped polygon in red inside the window.
8. Map every vertex of the clipped polygon to the viewport using the window to viewport transformation and draw it in the viewport.
9. When an arrow key is pressed, move the window and display the new clipped result.
10. When the key r is pressed, clear the polygon. When Esc is pressed, exit the program.
11. Stop.

The clipped polygon has four vertices, (51.03, 100), (175.17, 300), (224.83, 300) and (348.97, 100), and is drawn in red inside the window. The same polygon is drawn in the viewport at (451.03, 120), (575.17, 280), (624.83, 280) and (748.97, 120). The viewport is scaled by 1 in the x direction and by 0.8 in the y direction. (The coordinates are in pixels measured from the lower left corner of the window.) Pressing the arrow keys moves the clipping window and the clipped polygon changes accordingly. Pressing r clears the screen.

Result

The program accepts a polygon through mouse clicks, clips it against the window using the keyboard command, and displays the clipped polygon in the viewport.

Conclusion

Thus, polygon clipping with respect to a window and a viewport was successfully implemented using C++ and OpenGL with a mouse and keyboard interface. The Sutherland–Hodgman method keeps the clipped result as a closed polygon, and the window to viewport transformation maps it to the screen.

Practical No. 7

Title:

Implementation of 2-D and 3-D Transformations.

Aim

To implement the basic 2-D transformations (translation, scaling and rotation) on a triangle and the basic 3-D transformations (translation, scaling and rotation about the X, Y and Z axes) on a cube using homogeneous coordinates in C++ and OpenGL.

Objectives

1. To understand homogeneous coordinates and transformation matrices.
2. To perform translation, scaling and rotation of a 2-D object.
3. To perform translation, scaling and rotation of a 3-D object.
4. To display a 3-D object on a 2-D screen using an oblique projection.
5. To display the original object and the transformed object together.

Theory

A transformation changes the position, size or orientation of an object. In homogeneous coordinates a 2-D point (x, y) is written as (x, y, 1) and a 3-D point (x, y, z) as (x, y, z, 1). All the basic transformations then become matrix multiplications, P' = M × P, and several transformations can be combined into one matrix.

2-D Transformation Matrices (3 × 3)

- Translation by (tx, ty): [1 0 tx ; 0 1 ty ; 0 0 1]
- Scaling by (sx, sy) about the origin: [sx 0 0 ; 0 sy 0 ; 0 0 1]
- Rotation by angle θ about the origin: [cosθ -sinθ 0 ; sinθ cosθ 0 ; 0 0 1]

3-D Transformation Matrices (4 × 4)

- Translation by (tx, ty, tz): identity matrix with tx, ty, tz in the last column.
- Scaling by (sx, sy, sz): diagonal matrix with sx, sy, sz, 1.
- Rotation about the Z axis: x' = x cosθ - y sinθ, y' = x sinθ + y cosθ, z' = z.
- Rotation about the X axis: y' = y cosθ - z sinθ, z' = y sinθ + z cosθ, x' = x.
- Rotation about the Y axis: z' = z cosθ - x sinθ, x' = z sinθ + x cosθ, y' = y.

Oblique Projection

The screen is two-dimensional, so the 3-D cube is drawn using an oblique (cabinet) projection: xp = x + 0.5 z cos45° and yp = y + 0.5 z sin45°. The 12 edges of the cube are drawn between the projected vertices.

Algorithm

1. Start the program and select 2-D or 3-D transformation.
2. Define the original object: a triangle with vertices (50, 50), (150, 50), (100, 130) for 2-D, and a cube of side 100 with one corner at the origin for 3-D.
3. Read the type of transformation (translation, scaling or rotation) and its parameters.
4. Build the 3 × 3 matrix (2-D) or the 4 × 4 matrix (3-D) for the selected transformation.
5. Multiply every vertex (in homogeneous form) by the matrix to get the transformed vertex.
6. Display the old and new vertex coordinates.
7. Create the graphics window and draw the X-axis and Y-axis.
8. Draw the original object in blue and the transformed object in red (3-D objects are drawn using the oblique projection).

Result

The program performs translation, scaling and rotation on a 2-D triangle and a 3-D cube using matrix multiplication, and displays the original and the transformed objects.

Conclusion

Thus, 2-D and 3-D transformations were successfully implemented using C++ and OpenGL. Homogeneous coordinates allow translation, scaling and rotation to be written as matrix multiplications, and an oblique projection is used to display the 3-D object on the screen.

Practical No. 8

Title:

Implementation of 2-D Transformations on an Object with Respect to the Axes: (i) Scaling, (ii) Rotation about an Arbitrary Point, (iii) Reflection.

Aim

To apply scaling, rotation about an arbitrary point and reflection on a 2-D object entered by the user, using composite transformation matrices in C++ and OpenGL.

Objectives

1. To represent 2-D transformations using 3 × 3 homogeneous matrices.
2. To perform scaling of an object about a fixed point.
3. To perform rotation of an object about an arbitrary pivot point using a composite matrix.
4. To perform reflection about the X-axis, the Y-axis, the origin and the lines y = x and y = -x.
5. To display the original and the transformed object with respect to the X and Y axes.

Theory

In homogeneous coordinates a point (x, y) is written as (x, y, 1) and every 2-D transformation is a 3 × 3 matrix. The matrices of several transformations can be multiplied to get one composite matrix.

Scaling

Scaling by sx and sy about the origin uses the matrix [sx 0 0 ; 0 sy 0 ; 0 0 1]. To scale about a fixed point (xf, yf), so that this point does not move, the composite matrix is M = T(xf, yf) · S(sx, sy) · T(-xf, -yf).

Rotation about an Arbitrary Point

The rotation matrix R(θ) = [cosθ -sinθ 0 ; sinθ cosθ 0 ; 0 0 1] rotates an object about the origin. To rotate about a pivot point (px, py), three steps are combined:

- Translate the pivot point to the origin: T(-px, -py).
- Rotate about the origin: R(θ).
- Translate the pivot point back: T(px, py).
- Composite matrix: M = T(px, py) · R(θ) · T(-px, -py).

Reflection

Reflection produces a mirror image of the object. The matrices are:

- About the X-axis: [1 0 0 ; 0 -1 0 ; 0 0 1]
- About the Y-axis: [-1 0 0 ; 0 1 0 ; 0 0 1]
- About the origin: [-1 0 0 ; 0 -1 0 ; 0 0 1]
- About the line y = x: [0 1 0 ; 1 0 0 ; 0 0 1]
- About the line y = -x: [0 -1 0 ; -1 0 0 ; 0 0 1]

Algorithm

1. Start the program.
2. Read the number of vertices and the coordinates of the object.
3. Display the menu: scaling, rotation about an arbitrary point, reflection.
4. For scaling: read sx, sy and the fixed point, and calculate M = T(xf, yf) · S · T(-xf, -yf).
5. For rotation: read the angle and the pivot point, and calculate M = T(px, py) · R · T(-px, -py).
6. For reflection: read the type of reflection and select the corresponding matrix.
7. Multiply every vertex (x, y, 1) of the object by M to get the new vertex.
8. Print the composite matrix and the new coordinates.
9. Create the window, draw the X-axis and Y-axis with the origin at the centre, and draw the original object in blue and the transformed object in red.
10. Stop.

Result

The program performs scaling, rotation about an arbitrary point and reflection on a 2-D object using composite transformation matrices, and displays the original and transformed objects.

Conclusion

Thus, scaling, rotation about an arbitrary point and reflection were successfully implemented using C++ and OpenGL. Representing the transformations as homogeneous matrices makes it easy to combine translation, rotation and scaling into a single composite matrix.